Department of Artificial Intelligence and Data Science	
Class and Semester	S/A1, Semester III, AY 2026 to 27
Name	Aaryan Nandkishor Raorane
Roll No and SAP ID	S004, 60018250070
Experiment	Multithreading in Python using the threading module

1. Aim

To create and run two threads using the threading module of Python, to observe their output, and to compare the execution with a program that performs the same tasks sequentially.

2. Procedure

The threading module was imported and two functions were defined to perform different tasks. Two thread objects were created using threading. Thread(), both were started with the start() method, and the join() method was used to make the main program wait until both had completed. The same two tasks were then run one after the other in a separate program, and the total time of both versions was noted.

3. Program Using Threads

```
import threading
import time

def print_numbers():
    for i in range(1, 6):
        print("Number: " + str(i))
        time.sleep(1)

def print_letters():
    for letter in "ABCDE":
        print("Letter: " + letter)
        time.sleep(1)

start_time = time.time()

t1 = threading.Thread(target=print_numbers)
t2 = threading.Thread(target=print_letters)

t1.start()
t2.start()

t1.join()
t2.join()
```

```
end_time = time.time()
print("Total time with threads: " + str(round(end_time - start_time, 2)) + " seconds")
```

Output

```
Number: 1
Letter: A
Number: 2
Letter: B
Number: 3
Letter: C
Number: 4
Letter: D
Number: 5
Letter: E
Total time with threads: 5.0 seconds
```

Both tasks ran at the same time, so the numbers and letters appear alternately. Each task takes five seconds on its own, and the total time stayed close to five seconds.

4. Program Using Sequential Execution

```
import time

def print_numbers():
    for i in range(1, 6):
        print("Number: " + str(i))
        time.sleep(1)

def print_letters():
    for letter in "ABCDE":
        print("Letter: " + letter)
        time.sleep(1)

start_time = time.time()

print_numbers()
print_letters()

end_time = time.time()
print("Total time sequentially: " + str(round(end_time - start_time, 2)) + " seconds")
```

Output

```
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
Letter: A
Letter: B
Letter: C
Letter: D
Letter: E
Total time sequentially: 10.0 seconds
```

The second task could begin only after the first task had finished, so the total time was the sum of both, close to ten seconds.

5. Comparison of the Two Programs

Parameter	Threaded program	Sequential program
Order of output	Mixed, tasks overlap	Fixed, one task after the other
Total time	About 5 seconds	About 10 seconds
Waiting time	Used by the other thread	Wasted, nothing else runs
Predictability	Order can change on each run	Same order on every run

6. Questions and Answers

Q1. What is multithreading in Python?

Multithreading is the technique of running more than one thread of execution inside a single process. A thread is the smallest unit of work that the operating system can schedule, and all threads of a process share the same memory. In Python it is used to let several tasks make progress during overlapping time, for example one task waiting for a file while another keeps working. The threading module provides this facility.

Q2. What is the purpose of the threading module?

The threading module gives Python programs a simple way to create, start and manage threads. Its Thread class is used to build a thread from a function, and methods like start() and join() control how that thread runs and finishes. The module also provides Lock objects, which protect shared data when two threads try to change it together.

Q3. Differentiate between start() and join().

Point	start()	join()
Purpose	Begins the execution of a thread	Makes the calling thread wait until another thread finishes
Effect on main program	Returns at once, main program continues	Main program stops and waits at that line
Number of calls	Only once per thread, a second call raises RuntimeError	Can be called any number of times
Typical use	To launch a task in the background	To make sure a task is complete before moving ahead

```
import threading

def show():
    print("Thread is running")

t = threading.Thread(target=show)
t.start()
t.join()
print("Main program ends after the thread")
```

Output

```
Thread is running
Main program ends after the thread
```

Q4. Why can the order of output vary between executions?

The order is decided by the operating system scheduler and not by the Python program. The scheduler switches the processor between threads at moments that depend on the system load, on other running programs and on tiny timing differences, such as when each sleep() call ends. Because these conditions are never exactly the same twice, two threads that print at nearly the same instant can appear in a different order, or even have their lines mixed on one line, on different runs. In our own runs of the threaded program, the letter A appeared before Number 1 in one execution and after it in another.

Q5. What is the difference between a process and a thread?

Point	Process	Thread
Meaning	An independent program in execution	A smaller unit of execution inside a process
Memory	Has its own separate memory space	Shares the memory of its parent process

Point	Process	Thread
Creation cost	Heavy, takes more time and resources	Light, quick to create and switch
Communication	Needs special methods such as pipes or queues	Can use shared variables directly
Failure effect	A crash does not affect other processes	A crash can bring down the whole process
Python module	multiprocessing	threading

Q6. Mention two real world applications of multithreading.

1. Web servers. A server handles many client requests at once by giving each request its own thread, so one slow client does not hold up the others.

2. Graphical applications. A word processor uses separate threads for typing, spell checking and automatic saving, which keeps the window responsive while background work continues.

Q7. Does multithreading always improve program execution time? Justify your answer.

No, it does not. Multithreading helps when a program spends most of its time waiting, such as for network replies, file operations or sleep(), because another thread can use that waiting time. This is why our threaded program finished in about five seconds instead of ten.

For tasks that keep the processor busy with calculations, the standard Python interpreter does not give a speed gain. The Global Interpreter Lock allows only one thread to execute Python code at a time, so two threads simply take turns. Creating threads and switching between them also adds a small overhead, and shared data needs locks, which can slow the program further. The following test runs a heavy counting loop twice, first sequentially and then in two threads.

```
import threading
import time

def count(n):
    total = 0
    for i in range(n):
        total += i

N = 20000000

start_time = time.time()
count(N)
count(N)
print("Sequential: " + str(round(time.time() - start_time, 2)) + " seconds")

start_time = time.time()
```

```
t1 = threading.Thread(target=count, args=(N,))
t2 = threading.Thread(target=count, args=(N,))
t1.start()
t2.start()
t1.join()
t2.join()
print("Threaded: " + str(round(time.time() - start_time, 2)) + " seconds")
```

Output

```
Sequential: 2.25 seconds
Threaded: 2.2 seconds
```

The two timings are almost equal, which shows that threads did not reduce the time for processor bound work. Such work is better handled using the multiprocessing module.

7. Conclusion

Two threads were created with the threading module, started with start() and synchronised with join(). The threaded program completed in about half the time of the sequential program because the waiting periods of the two tasks overlapped. Multithreading is therefore useful for waiting type tasks, but it does not speed up heavy calculations in Python.